

Expense Tracker — Technical Documentation

Stack: Python · FastAPI · PostgreSQL · SQLAlchemy · Pydantic · Pytest

1. Project Overview

The Smart Expense Tracker is a RESTful API that allows users to create, read, update, and delete personal expenses, and generate spending reports. It is built as a **production-style** backend application demonstrating clean layered architecture, input validation, database integration, and automated testing.

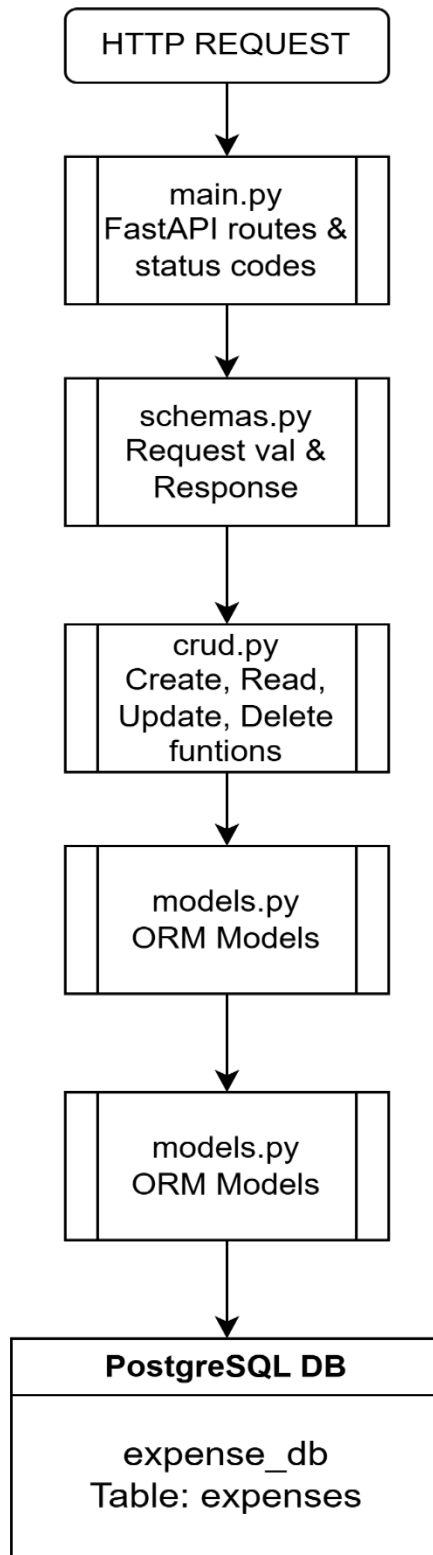
Functions

- Full CRUD operations on records
- Bulk creation of multiple expenses in one request
- Reporting summary: shop, weekly, monthly, yearly totals
- Automated tests
- PostgreSQL as the database

2. Architecture

Layered Architecture

The application is split into 5 different layers, each with a single responsibility:



3. File-by-File Breakdown

3.1 database.py

Manages the database connection. Everything related to the database starts here.

Component	Function
DATABASE_URL	Reads connection string from environment variable
create_engine()	Creates the SQLAlchemy engine — the core connection pool to PostgreSQL
sessionmaker()	Factory that produces DB sessions with autocommit=False means you control when data is saved
Base	Base class all ORM models inherit from
get_db()	A generator function used as a FastAPI dependency

yield in get_db():

Request comes in → session opens → yield hands session to route →

route uses session → finally block runs → session closes

This is called the context manager pattern. It guarantees cleanup even when exceptions occur.

3.2 models.py

Defines what the database table looks like using Python classes.

Table name = "expenses"

Columns: id, description, amount, date

Component	Function
class Expense(Base)	Inherits from Base — registers this class as a DB table
tablename	The actual table name in PostgreSQL
Column(Integer, primary_key=True)	Auto-incrementing unique ID for each row
index=True	Creates a DB index on id — makes lookups by ID faster
nullable=False	Enforces that this column cannot be empty at the DB level
Column(Float)	Stores decimal numbers (e.g. 45.50)
Column(Date)	Stores date only (no time) — e.g. 2026-03-27

SQLAlchemy ORM :

Instead of writing raw SQL like INSERT INTO expenses VALUES (...), we use Python:

```
expense = Expense(description="Walmart", amount=45.50, date=date.today())
```

```
db.add(expense)
```

```
db.commit()
```

SQLAlchemy translates this to SQL automatically.

3.3 schemas.py

Defines the shape of data coming IN (requests) and going OUT (responses). Uses Pydantic for automatic validation.

Schema	Function
ExpenseBase	Shared fields — parent class amount gt=0 means amount must be greater than zero
ExpenseCreate	POST /expenses request body
ExpenseUpdate	PATCH /expenses/{id} request body
ExpenseOut	Every response that returns an expense

Two schemas for input and output

- Input (ExpenseCreate) has no id - the DB generates it
- Output (ExpenseOut) includes id - the user needs to know it

from_attributes=True- without this, you cannot convert a SQLAlchemy ORM object (from the DB) directly into a Pydantic schema. This one line makes that conversion automatic.

3.4 crud.py

Contains all database operations. Zero FastAPI imports - pure Python + SQLAlchemy.

3.5 main.py

The FastAPI application. Defines all HTTP routes and connects them to the CRUD layer.

3.6 tests/conftest.py

Shared test setup. Provides fixtures - reusable components that every test can use.

The rollback pattern - test isolation :

Test starts → transaction opens → test runs (adds/deletes data) → test ends → transaction.rollback() → DB back to clean state → next test starts fresh

Every test gets a perfectly clean database without needing to delete data manually.

app.dependency_overrides- FastAPI's built-in mechanism to replace dependencies at runtime. During tests, get_db is replaced with a function that returns the test SQLite session instead of the real PostgreSQL session.

3.7 tests/test_expenses.py

Tests all 6 CRUD endpoints with both happy path and edge cases.

Test coverage by class:

Test Class	Tests
TestCreateExpense	Creates successfully, rejects zero/negative amount, empty description, missing fields
TestBulkCreate	Creates multiple, rejects empty list, rejects invalid item
TestListExpenses	Returns empty list, returns created items, pagination skip, pagination limit
TestGetExpense	Returns correct expense, 404 for missing
TestUpdateExpense	Updates amount, updates description, rejects negative on update, 404 for missing
TestDeleteExpense	Deletes successfully, deleted item is gone, 404 for missing

3.8 tests/test_reports.py

Tests all the reporting endpoints including boundary conditions.

test scenarios:

Test	Functionality
test_groups_by_shop	Two Walmart entries sum correctly; Target is separate
test_ordered_by_total_descending	Results sorted by highest spent first
test_includes_expense_from_today	Today's date is within the 7-day window
test_excludes_expense_older_than_7_days	Day 8 is correctly excluded
test_different_month_excluded	The expenses of one month don't appear in another month's report
test_rejects_invalid_month	Invalid month returns 400 Bad Request

4. Data Flow Lifecycle

1. HTTP POST arrives at FastAPI
2. FastAPI finds the matching route in main.py
3. Pydantic (schemas.py) validates the request body
 - amount must be > 0
 - description must not be empty
 - date must be a valid date
 - If invalid: return 422 immediately, never reach DB
4. FastAPI calls Depends(get_db) → opens a DB session
5. Route function calls crud.create_expense(db, expense)
6. crud.py creates a models.Expense ORM object
7. db.add() → db.commit() → data saved to PostgreSQL
8. db.refresh() → re-fetches the row to get generated id
9. ORM object returned to route in main.py
10. FastAPI serialises it using ExpenseOut schema → only fields defined in ExpenseOut are returned
11. HTTP 201 response sent to client
12. Finally block in get_db() closes the DB session

-----*

Below are the steps for setup and perform basic functions.
Download the requirements and python files and store it in a folder.
Install the PgAdmin for databases and PostgreSQL to view the data.

Terminal 1 — Start the server

Step 1. Navigate to your project folder

```
cd C:\Users\mohan\OneDrive\Desktop\Python
```

Step 2. Activate the virtual environment

```
.venv\Scripts\activate
```

Step 3. Set the database URL(replace **password** with your password entered during the PgAdmin setup) in the terminal to connect to the database

```
$env:DATABASE_URL="postgresql://postgres:password@localhost:5432/expense_db"
```

Step 4. Start the server

```
uvicorn expense_tracker.main:app --reload
```

You should see:

```
INFO: Uvicorn running on http://127.0.0.1:8000
```

```
INFO: Application startup complete.
```

```
---
```

Browser: Demo the API

Step 5. Open Swagger UI

```
http://127.0.0.1:8000/docs
```

Step 6. Add an expense via POST /expenses → Try it out:

```
json
```

```
{  
  "description": "Nike",  
  "amount": 120.00,  
  "date": "2026-03-28"  
}
```

Step 7. Add a few more for the reports to look interesting:

```
json
```

```
{ "description": "Starbucks", "amount": 6.50, "date": "2026-03-28" }
```

json

```
{ "description": "Nike", "amount": 80.00, "date": "2026-03-27" }
```

json

```
{ "description": "Amazon", "amount": 250.00, "date": "2026-03-26" }
```

Step 8. View all expenses via GET /expenses → Try it out → Execute

Step 9. Check reports:

- GET /reports/summary → shows totals per shop
- GET /reports/weekly → shows last 7 days total
- GET /reports/monthly → use month=3&year=2026
- GET /reports/yearly → use year=2026

Step 10. Update an expense via PATCH /expenses/{id}:

json

```
{ "amount": 150.00 }
```

Step 11. Delete an expense via DELETE /expenses/{id} → enter an id → Execute

Terminal 2 — Run the tests

Open a second terminal and run:

Step 12. Navigate to project

```
cd C:\Users\mohan\OneDrive\Desktop\Python
```

Step 13. Activate venv

```
.venv\Scripts\activate
```

Step 14. Run all 35 tests

```
pytest expense_tracker/tests/ -v
```

Step 15. Run just one specific test

```
pytest
```

```
expense_tracker/tests/test_expenses.py::TestCreateExpense::test_creates_successfully -v
```

Step 16. Run just the reports tests

```
pytest expense_tracker/tests/test_reports.py -v
```